

Greatwheel

An rLM-Coordinated Agentic Runtime:
System Overview and Experimental Findings

M. Berends

March 2026

Abstract

Greatwheel is an agentic runtime built on the *Recursive Language Model* (rLM) paradigm: language models write Python code in sandboxed REPLs, using variables—not the context window—as their working memory. A Rust core manages execution, routing, and observability while Python agents run inside `ouros` sandboxes. This report describes the system architecture, summarises experiments conducted on the BrowseComp-Plus factoid retrieval benchmark, and outlines planned next steps.

Introduction

Most agentic systems pass retrieved context directly into an LLM’s context window. Greatwheel takes a different approach: the model never sees full context. Instead, every external operation—LLM calls, memory queries, inter-agent messages—is a *pause point* where the Rust runtime intercepts, validates permissions, records traces, and resumes execution. The agent’s working state lives as Python variables that persist across turns, giving the system perfect auditability and fine-grained resource control.

A “bare agent” with no predefined code is still functional: the rLM can compose SDK globals (`llm`, `memory`, `agent`, `channel`) into ad-hoc tools on the fly, and those tools survive to the next turn because they are ordinary Python functions in a persistent session.

Architecture

The system is organised as a nine-crate Rust workspace with the following responsibilities:

gw-core	Shared types: IDs, Task, AgentDef, CallContext, permissions.
gw-runtime	<code>ouros</code> integration, REPL session management, host-function bridge.
gw-llm	LLM provider trait and Ollama client (via <code>rl-play</code> proxy).
gw-memory	Hybrid search: LanceDB vectors + tantivy BM25 + Postgres, with RRF fusion.
gw-bus	Inter-agent message bus (call/notify).
gw-channels	Channel adapters (HTTP, WebSocket, webhooks).
gw-scheduler	Task queue, per-org concurrency, rate limiting.
gw-trace	OpenTelemetry GenAI instrumentation with Postgres export.
gw-server	Binary crate: HTTP/WS API that wires everything together.

The production stack runs four Docker Compose services: the Greatwheel binary, `rl-play` (an LLM proxy that captures conversations for fine-tuning), Postgres, and Ollama.

The rLM Execution Loop

1. A user message creates a **Task** dispatched to an agent’s **ouros** session.
2. Context is injected as a Python variable (`_task = {"payload": ...}`).
3. The rLM receives a system prompt, a truncated session summary, and the user message. It outputs Python code.
4. **ouros** executes the code. Pure computation runs immediately; host-function calls (`llm`, `memory`, etc.) *pause* the session, and the Rust runtime fulfils each call before resuming.
5. The loop repeats until the rLM produces a `FINAL()` response or an iteration limit is reached.
6. All Python variables persist to the next conversational turn.

Observability and Fine-Tuning

Every operation emits OpenTelemetry spans following GenAI semantic conventions. Spans are exported to the console, OTLP, and Postgres (the latter enables fine-tuning dataset extraction). All LLM calls also flow through `rl-play`, which groups conversations by session and supports SFT, DPO, and PPO exports—creating a closed loop from production traces to improved models.

BrowseComp-Plus Experiments

BrowseComp-Plus is a factoid retrieval benchmark: 30 queries over a 100K web document corpus. It serves as the primary testbed for our rLM + retrieval research. We have conducted over 40 experiment runs, methodically varying retrieval strategy, prompting, and model configuration.

Current Best Result

Our best configuration achieves **36.7% (11/30)** with a reproducibility range of 9–11 across runs (mean ≈ 10). Per-query timing averages 115s, of which 90% is LLM inference in the rLM loop and 10% is pre-search (decomposition + BM25 + PRF).

What Has Worked

Retrieval.

- **Boosted BM25:** Phrase match ($4\times$), slop phrase ($2\times$), AND’d terms ($1.5\times$), OR’d terms ($1\times$). This was the single largest contributor, lifting accuracy by 2–3 queries.
- **Pseudo-relevance feedback (PRF):** Extracting distinctive terms from round-1 snippets for expanded queries added ~ 1 –2 queries.
- **Multi-query decomposition:** Breaking multi-hop questions into independent sub-fact searches ($1 \rightarrow 5$ queries) meaningfully broadened retrieval coverage.
- $k = 10$ **retrieval breadth:** More documents per search without overwhelming the model.

Prompting and loop control.

- **Document-grounded prompts:** Forcing `get_document()` + `llm_query()` and emphasising corpus-only answers.
- **VERIFY directive:** “Search for the candidate answer by name to confirm” catches extraction errors.
- **12 iterations:** Enough to explore, verify, and submit. 16 iterations caused second-guessing.
- **think:false for utility calls:** Saves tokens on pre-search extraction and fallback.

- **Fallback extraction:** Recovers answers when the model does not call `FINAL()` in time.

What Has Not Worked

- **HyDE (hypothetical document embeddings):** No benefit for BM25 queries.
- **Dense vector search** (all six variants tested): Added zero lift to BM25-only. Variants included asymmetric prefixes, BM25-weighted RRF fusion, increased IVF-PQ probes, and a fully re-embedded corpus. BrowseComp queries are entity-heavy (names, dates, places) where BM25 keyword matching naturally excels; 768-dimensional dense vectors (`nomic-embed-text`) lack discrimination at 100K scale.
- **Best-of- N majority voting:** Wrong answers agreed more often than correct ones— $3\times$ cost, same accuracy.
- **Wider pre-search** (8+5 queries): Overwhelmed the model with context.

Key Insight: The Retrieval Ceiling

86% of failures are retrieval misses—the relevant documents are never found. The 9B model reasons well once it has the right documents; the bottleneck is search, not extraction. However, the missing documents often lack both keyword *and* semantic overlap with the query. They require reasoning about *what kind of document would contain the answer*, which neither BM25 nor dense retrieval performs.

Challenges

Beyond the retrieval ceiling, several broader challenges have shaped development:

1. **High per-query variance.** The same configuration scores 9–11 out of 30 across runs. Individual queries flip between correct and incorrect nondeterministically, making it hard to distinguish real gains from noise.
2. **Token cost.** Each query consumes $\sim 150\text{K}$ tokens across a 12-iteration loop. The rLM loop dominates wall-clock time, and reducing iterations hurts accuracy.
3. **Incomplete runtime.** Several crates (`gw-bus`, `gw-scheduler`, `gw-channels`) have trait definitions but lack concrete implementations. The Python-side agent SDK is designed but not yet wired to the host bridge.
4. **Multi-tenancy.** Core types support org/user/session hierarchy and per-agent permissions, but auth middleware and session lifecycle (idle timeout, snapshot, eviction) are not yet implemented.

Next Steps

Our roadmap separates retrieval-quality improvements from orchestration and infrastructure work.

Near-Term: Retrieval

ColBERT reranking (R2)

Use Reason-ModernColBERT (149M params) as either a primary retriever or a BM25 reranker. ColBERT’s late-interaction token-level MaxSim scoring has demonstrated strong results on BrowseComp-Plus in external work. We plan three modes: ColBERT-only, BM25 + ColBERT RRF fusion, and BM25 $\rightarrow$ ColBERT rerank. Requires $\sim 10\text{--}16$ GB VRAM.

Larger rLM model (A3)

Use a 32B model (`qwen2.5:32b` or `qwen3.5:32b`) for the main rLM loop, keeping the 9B model for utility calls. Better reasoning should produce better search queries and extraction.

Ensemble (A4)

Run 2–3 diverse configurations per query and take the union of correct answers. Historical analysis suggests the union across existing configs reaches ~40%.

Near-Term: Optimisation Infrastructure

GEPA integration

Use GEPA’s `optimize_anything` API for systematic prompt and parameter optimisation. The key unlock is *actionable side information* (ASI): per-query failure-mode diagnostics that tell the optimiser *why* a run scored as it did, not just the scalar accuracy.

ASI infrastructure

Even without GEPA, structured per-query diagnostics (failure mode, retrieval recall, searches issued, tokens consumed) enable smarter manual iteration.

Medium-Term: Runtime Completion

- **Episodes:** Compressed summaries of agent work that flow as context between agents, enabling composability and multi-agent collaboration.
- **Agent SDK:** Wire Python-side `llm`, `memory`, `agent`, `channel` globals to the host bridge.
- **Session lifecycle:** Idle timeout, snapshot to Postgres, eviction.
- **Scheduler and bus:** Concrete implementations of task queuing, rate limiting, and inter-agent messaging.

Conclusion

The core architectural bet—context as Python variables, not context window—has proven sound. The rLM + ouros execution model delivers auditability, composability, and resource control that would be difficult to achieve in a purely prompt-based system. On BrowseComp-Plus, boosted BM25 with pseudo-relevance feedback and document-grounded prompting reaches 37% accuracy, with clear evidence that the retrieval ceiling (not model reasoning) is the binding constraint. ColBERT reranking and larger models are the most promising levers for the next phase of work.